Shopper Context

Shopper Context enables personalized commerce experiences by associating session-level attributes–such as source codes, customer groups, custom qualifiers, and assignment qualifiers–with a shopper's session. These attributes influence pricing, promotions, product sorting, and other backend behaviors, and you don't need to change individual API calls.

This integration wraps the Shopper Context API and manages qualifier state through server middleware, cookies, and a React hook.

Feature Flag

By default, Shopper Context is disabled. Turn it on with an environment variable or configuration:

```
PUBLIC__app__features__shopperContext__enabled=true
```

When Shopper Context is off, the middleware exits early and doesn't call Salesforce Commerce API (SCAPI).

How It Works

Qualifiers enter the system through two paths:

1. URL query parameters: The server middleware extracts qualifiers from the URL on every request (for example, `?src=spring-sale&deviceType=mobile`).
2. React hook: The `useShopperContext` hook lets components update qualifiers from UI interactions (for example, store selector, coupon input).

Both paths merge new qualifiers with the current state stored in cookies, then send the full merged context to SCAPI with a PUT request that fully replaces the existing context. The integration writes the updated state back to cookies for subsequent requests.

Qualifier Support

The template supports the following qualifiers from the Shopper Context API out of the box:

		and promotion targeting
`couponCodes`	`couponCodes`	Comma-separated list of coupon codes
`customQualifiers`	`deviceType`	For example, `?deviceType=mobile`
`assignmentQualifiers`	`store`	For example, `?store=boston`

`effectiveDateTime`, `customerGroupIds`, `clientIp`, and `geoLocation` aren't enabled by default. To turn them on, update `SHOPPER_CONTEXT_SEARCH_PARAMS` in `packages/template-retail-rsc-app/src/lib/shopper-context/constants.ts`. Verify that the API supports the qualifier–see the ShopperContext type reference. The `geoLocation` qualifier is a structured object, so it typically needs additional handling.

To add a validation layer on commerce API requests, consider the Shopper Context Hooks cartridge. It intercepts calls to create/update context and validates the body against a predefined scope for a given client ID.

Cookies

Shopper Context uses two `httpOnly` cookies to persist qualifier state on the server. The server reads qualifier state from the cookies, so the integration doesn't refetch the full context from SCAPI on every request. The middleware also includes those qualifiers in subsequent GET calls without extra API round-trips.

Cookie	Base Name	Default Expiry	Purpose
Shopper Context	`storefront-next-context`	6 hours	All qualifiers except source code
Source Code	`dwsourcecode`	30 days	Source code qualifier

On each request, the middleware reads current state from cookies, merges in any new qualifiers, sends the merged context to SCAPI if anything changed, and writes updated cookies back. New values overwrite existing keys. The merge preserves unmentioned keys.

Customization

To add a new qualifier, edit `SHOPPER_CONTEXT_SEARCH_PARAMS` in `packages/template-retail-rsc-app/src/lib/shopper-context/constants.ts`. After you add a qualifier, the template

URL parameters and updates SCAPI and cookies without a full page navigation.

```
import { useShopperContext } from "@/hooks/use-shopper-co

function StoreSelector() {
  const { updateQualifiers, isLoading, error, success } =

  const handleStoreChange = (storeId: string) => {
    updateQualifiers({ store: storeId });
  };

  return (
    <select onChange={(e) => handleStoreChange(e.target.v
      <option value="boston">Boston</option>
      <option value="nyc">New York</option>
    </select>
  );
}
```

Expiry

- Browser cookies: The storefront clears these cookies when the shopper logs out or if authentication fails. The shopper context cookie expires after 6 hours, and the source code cookie expires after 30 days. Cookie expiry values are defined in `packages/template-retail-rsc-app/src/lib/shopper-context/constants.ts`.
- API-side context: The API ties context to the USID (Unique Shopper ID). The API doesn't currently notify the storefront when the backend clears context, so the storefront can't detect the change. The next request re-creates the context when the shopper sets new qualifiers.

Try for free